PersistentTyping: design review of the whole project

Repository `CU-NVM/PersistentTyping` @ `64bb1ab` · evidence from runs of the current, unmodified tool on ecee-bilbo (Clang 20.0.0git, PMDK 1.11.1) · 2026-10-19 · **revised 2026-10-20 after design discussion**

> **How to read this.** §1–§3 describe the system as it stands: the idea, what "persistent" means in it and at which granularity, and what the Clang tool does. §4 states the properties a correctly persistified program must have, and checks each one against small probe programs run through the real tool. §5 lists the gaps that follow from that, ordered by how dangerous each one is. §6 proposes how to make the tool complete. §7 is the set of research decisions that only you can make. §8–§11 give my assessment of the idea, a comparison with existing systems, measured overheads, and where it could be published. §12 records the decisions from the 2026-10-20 discussion and the revised plan. Evidence tags: [probe] = run on bilbo for this review; [code] = read in the source; [HANDOFF] = recorded in HANDOFF.md; [inferred] = my reasoning, not yet tested; [demo] = plain-C++ stand-in compiled and run locally; [decided 2026-10-20] = decided in the 2026-10-20 discussion.

> **What changed in this revision.** The design discussion settled most of §7 and changed the type translation:
>
> - `pmem_string` is dropped. P(T) is now defined purely by the structure of a type, with `char*` handled like any other pointer to a scalar (§12.3).
> - Two silent bugs were confirmed with local demos: the constructor rewrite (§12.5) and the compiler-generated copy assignment of `persistent<F>`. You have fixed the second one locally.
> - The target is YCSB, there is a single writer per object, mismatched pools are refused, and pointers have new names (§12.2).
>
> Text in §1–§11 is unchanged except where marked [decided 2026-10-20]; §12 records the decisions and the revised plan.

1 · The idea

The user writes ordinary C++ classes and marks only the *entry points* of persistent data:

```
persistent<Stack>* s = new persistent<Stack>(0);   // at file scope or in main()
```

The tool then works out everything that must also become persistent, which is every class reachable from `Stack` through its fields. It generates a persistent version of each such class, and rewrites the program so that the data lives in a PMDK pool, survives restarts, and stays consistent after crashes. This is the same move as `numa<T,k>`: one annotation on the outer type, and *introspective typing* pushes it recursively into every sub-object.

The difference is what the annotation has to guarantee. NUMA placement is a performance property: if one node lands on the wrong socket, the program is slower but still correct. Persistence is a correctness property with three parts, and the tool has to deliver all three for every byte it touches:

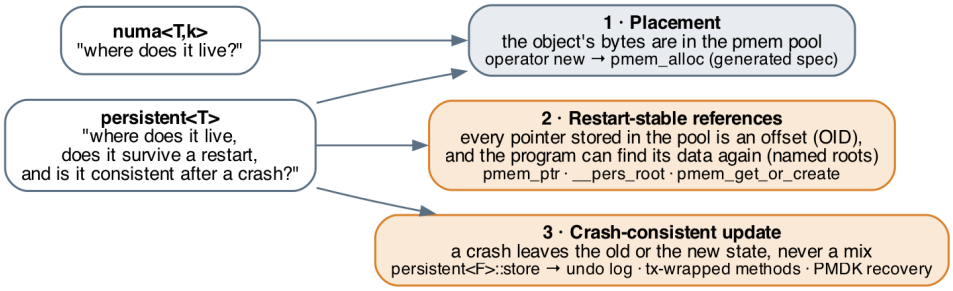


Figure 1. NUMA typing has to discharge one obligation; persistent typing has three. Obligations 2 and 3 (orange) have no NUMA counterpart, and that is where almost every gap in §5 comes from. The library pieces that discharge each obligation are listed in small type.

What you write, and what you get

This is the `stack` benchmark exactly as the current tool transforms it (output copied from the bilbo run). The original classes stay in the file unchanged; the tool appends one specialization per persistent type and rewrites the root.

user code — classes

```
class Node {
public:
    int   value;
    Node* next;
    Node(int v) : value(v), next(nullptr) {}
};

class Stack {
public:
    Node* top  = nullptr;
    int   size = 0;
    Stack(int size = 0) : size(size) {}
    void push(int v) {
        Node* n = new Node(v);
```

generated — persistent<Stack> (persistent<

```
template<>
class persistent<Stack> {
public:
    pmem_ptr<persistent<Node>> top
    persistent<int> size;
    static void* operator new(std:
    static void  operator delete(v
    persistent(int size) {
        pmem::obj::transaction::ru
    size = size;
});
    }

    void push(int v) {
```

```
        n->next = top;
        top = n;
        size++;
    }

    void print() const {
        Node* cur = top;
        std::cout << "stack: ";
        while (cur) {
            std::cout << cur->value << " ";
            cur = cur->next;
        }
        std::cout << "\n";
    }
};
```

user code — main

```
int main() {
    persistent<Stack>* s = new persistent<Stack>(0);

    pmem::obj::transaction::run(pmem_pool(), [&]{
        s->push(10);
        s->push(20);
        s->push(30);
    });

    s->print();
    return 0;
}
```

```
        pmem::obj::transaction::ru
        persistent<Node>* n = new
        n->next = top.get();
        top = n;
        size++;
    });
    }

    void print() const {
        pmem::obj::transaction::ru
        persistent<Node>* cur = to
        std::cout << "stack: ";
        while (cur) {
            std::cout << cur->valu
            cur = cur->next.get();
        }
        std::cout << "\n";
    });
    }

};
```

generated — root and main

```
struct __pers_root {
    pmem_ptr<persistent<Stack>> s;
};

__pers_root* __root = pmem_root<__

int main() {
    persistent<Stack>* s = pmem_ge

    pmem::obj::transaction::run(pm
        s->push(10);
        s->push(20);
        s->push(30);
    });
    s->print();
    return 0;
}
```

Look closely at the generated constructor: `size = size;`. The initialiser `: size(size)` was turned into a plain assignment inside the body, where the parameter `size` *shadows* the field. So the statement assigns the parameter to itself, and the field is never set. It only looks right because the value passed here is 0. The default argument `= 0` was also dropped. This is gap G4 in §5.

[decided 2026-10-20] **Fix:** keep the assignments inside the transaction-wrapped body, but write `this->size = size;`, emitted in field-declaration order. Carry over default arguments and in-class initialisers. The demo in §12.5 reproduces all three effects.

2 · What "persistent" means: granularity

"Persistent" is really several decisions, each at its own granularity. The project makes all of them implicitly; this section makes them explicit, because every gap in §5 sits under one of them.

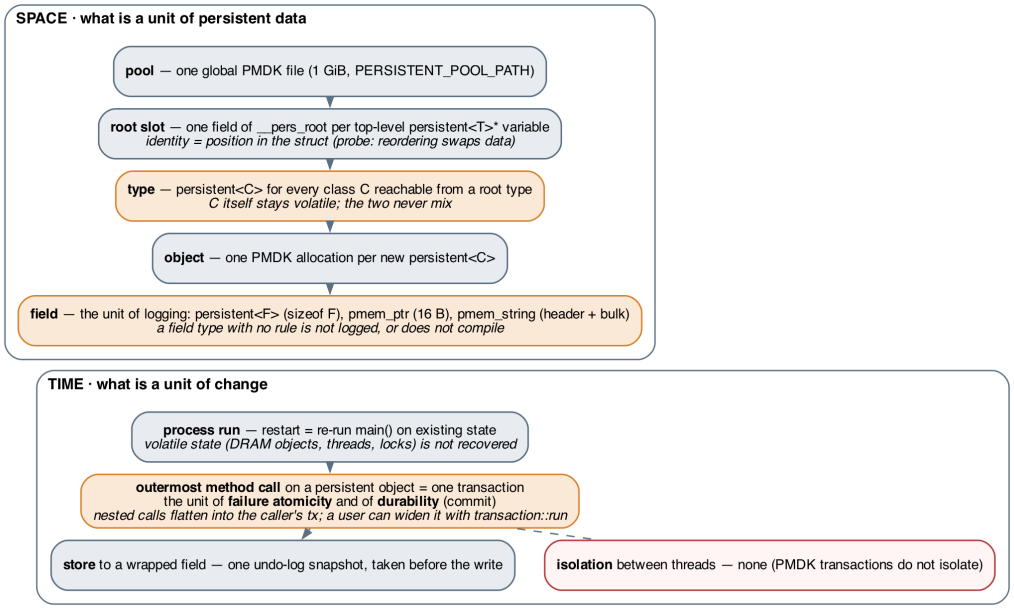


Figure 2. The granularities in the current design. Orange marks the two deliberate choices that define the programming model (persistence is per type; atomicity is per method call). Red marks a property the design does not provide at all.

question	current answer	consequence
What becomes persistent?	Every *type* statically reachable from a root type, through fields and through `new` in their methods. `persistent<C>` is a separate type from `C`. [code]	The same class can be used in DRAM and in pmem in one program. But a `persistent<C>*` cannot be passed where a `C*` is expected: the layouts differ, because a `pmem_ptr` is 16 bytes and a raw pointer 8. Anything not visible statically, such as `void*`, a base-class pointer, or an STL internal, is not reached.
Where does persistent data start?	Root variables: `persistent<T>* x = new persistent<T>(…)` at	A root's identity is its *position* in that struct, not its name. Reordering the declarations silently swaps the data [probe].

	file scope or in `main`, collected into `struct __pers_root`. [code]	
Unit of **allocation**	One PMDK object per `new persistent<C>`. All of them live in one global 1 GiB pool. [code]	No pool-level isolation between data sets; the pool size is a compile-time constant.
Unit of **logging**	One wrapped field. Every store first snapshots the old bytes into the undo log: `sizeof(F)` for a scalar, 16 bytes for a pointer, the header for a string. [code]	A field whose type has no wrapping rule is either not logged at all, or produces code that does not compile [probe].
Unit of **failure atomicity**	One *outermost* method call on a persistent object. Every generated method body is wrapped in `transaction::run`; nested calls flatten into the caller's transaction. [code]	Each YCSB operation (`insert`, `update`) is naturally one unit. Writes from outside any method are refused at run time [probe]. The user can widen the unit by wrapping several calls in their own `transaction::run`.
Unit of **durability**	The commit of the outermost transaction. [code]	Nothing is durable between commits. Read-only methods also open a transaction, because `print()` is wrapped too, so they pay its cost.
Unit of **recovery**	`pmemobj_open` rolls back the one unfinished transaction, then `main()` runs again, and `pmem_get_or_create` finds the existing roots. [probe]	Restart means re-running the program on existing state. Nothing volatile is recovered: DRAM objects, threads, locks, open files.
Unit of **isolation**	None. PMDK transactions provide atomicity and durability, not isolation, and the generated methods take no locks. [code]	Two threads calling methods on the same persistent object race, and a crash during the race can roll back one thread's work into the other's. YCSB is multi-threaded.

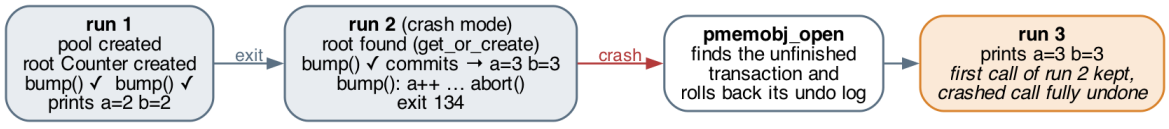


Figure 3. The atomicity and recovery semantics, measured. Probe 'atomic': Counter::bump() does a++, optionally abort(), then b++. Run 2's first bump() committed; its second crashed between a++ and b++. On restart, a and b agree (both 3): the crashed call was undone as a unit.

3 · What the Clang tool does

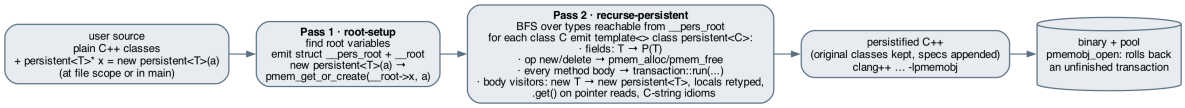


Figure 4. The two passes of persist-tool today. Pass 1 has no NUMA counterpart, because NUMA data never has to be found again after a restart. Pass 2 is the direct descendant of RecursiveNumaTyper.

1. **Pass 1, root-setup.** Finds every root variable (file scope or `main`). It emits `__pers_root`, with one `pmem_ptr` per root, and a global `__root = pmem_root<__pers_root>()`. It rewrites each `new persistent<T>(args)` into the find-or-create call `pmem_get_or_create(__root->x, args)`.
2. **Pass 2, recurse-persistent.** Walks the types reachable from `__pers_root` breadth-first. For each class `C` it emits a full `template<> class persistent<C>`:
    - every field gets the type P(T) from the table below;
    - `operator new` / `delete` route through the pool;
    - constructor initialiser lists become body assignments;
    - every method body is copied, rewritten by visitors (`new T` → `new persistent<T>`, locals retyped, `.get()` on pointer reads, C-string idioms), and wrapped in a transaction.
3. **Runtime library.** The library does the actual persistence work. `persistent<F>::store()` snapshots before writing; the mutable `operator T&` is deleted, so an unlogged scalar write cannot even be expressed, *except* through the compiler-generated copy assignment (`max = cur`), now fixed locally (§12.5). `pmem_ptr` stores an OID, i.e. {pool, offset}.

The tool's core is the type translation P(T), applied to every field:

field type T	P(T) today	correct?
`int`, `double`, … (fundamental F)	`persistent<F>`	✓ logged
`C` (user class by value)	`persistent<C>` + recursive specialization	✓
`U*` (pointer to user class)	`pmem_ptr<persistent<U>>`	✓ restart-stable, logged
`char*`, `std::string`	`pmem_string` (+ C-string idiom rewriting)	✓ when used as a string; ✗ for byte buffers (YCSB's payload). [decided 2026-10-20] **to be removed:** `char*` becomes an ordinary pointer

		to scalar (§12.3); `std::string` is a library class (out of scope)
`U**`, `int*`, any other pointer	`persistent<T*>`: a *raw address* stored in pmem, i.e. a **pmem-resident regular pointer** (§12.2)	✗ **meaningless after restart**
`int bins[4]` (fixed-size array)	`int[4] /* unwrapped */ bins;`	✗ invalid C++ [probe]. [decided 2026-10-20] support now as `persistent<int> bins[4]`
`std::vector<int>`, other library classes	`persistent<std::vector<int>> v;`, with no specialization found (tool prints "skipping std::vector<int>")	✗ **object in pmem, buffer in DRAM: run 2 segfaults [probe]. [decided 2026-10-20] out of scope for now: reject with an error**
base classes (`class D : public B`)	dropped	✗ `use of undeclared identifier 'id'` [probe]

> **Why arrays are a problem (since you asked).** A field `Node* next` is one pointer. The tool turns it into an offset into the pool, so it is still valid when the pool is mapped at a different address in the next run. An array of pointers, `Node** table`, is *two* levels of pointer: the field points to a block, and the block holds pointers. P(T) only has a rule for "pointer to a class". `Node**` is a pointer to a pointer, so it falls through to the generic rule and is stored as a raw address. Raw addresses are only meaningful inside the process that produced them, so run 2 follows a stale address. The same happens to `int* data = new int[n]`. Fixed-size arrays (`int bins[4]`) are the opposite case: there is no pointer at all, but there is also no rule, so the tool emits invalid C++. In short, arrays are not a special problem. They are one place where P(T) is *incomplete*, and §6 argues for fixing the completeness of P(T) in general rather than one type at a time.

4 · What a correct persistified program needs, and where the current design leaks

Seven invariants. If all seven hold, the program is crash-consistent and restartable. The matrix shows how each is enforced today, and the probe evidence.

#	invariant	enforced by	holes (evidence)

I1	**Placement.** Every object reachable from a root is in the pool.	generated `operator new` ; BFS over field types	Library classes are not specialized, and their internal buffers stay in DRAM: `std::vector` segfaults on run 2 [probe]. `new` outside a persistent method (e.g. in a free function) stays in DRAM [code].
I2	**Stable references.** Every pointer stored in the pool is an OID.	`pmem_ptr` for `U*`	`U**` , `int*` , `void*` , and pointers inside unwrapped types are stored raw [code].
I3	**Logged writes.** Every store to the pool happens inside a transaction, after a snapshot.	wrapped field types; deleted `operator T&` ; method transactions; runtime check	Stores into unwrapped field types (STL internals, unions, anything marked `/* unwrapped */`) bypass the log. `memcpy` / `memset` into an object bypasses it [inferred]. A write from outside any method is refused only at run time, by aborting [probe]. The compiler-generated copy assignment of `persistent<F>` skipped `store()` for `max = cur` and whole-object copies [demo]; fixed locally (§12.5).
I4	**No persistent → volatile links.** Nothing in the pool points to DRAM.	types: a `pmem_ptr` field only accepts `persistent<U>*`	A `persistent<U>` that is not in the pool (stack value, unspecialized fallback) gives `pmemobj_oid = OID_NULL` : a silent null [inferred, from PMDK documentation].
I5	**No leaks.** Every pool object is reachable from `__root` .	transactional alloc/ free (a crash never leaks)	Overwriting the last pointer to an object leaks it, as in plain C++, but the leak is *permanent* and survives every restart [inferred].
I6	**Schema agreement.** The program opening a pool has the same roots and layouts as the program that wrote it.	nothing	Reordering roots swaps the data (`a=1 b=2` → `a=2 b=1`). Adding a field reads garbage (`a=0 b=0`). No error in either case [probe].
I7	**Isolation.** Concurrent operations do not see each	nothing	Multi-threaded YCSB needs it. `pmem_get_or_create` also races [HANDOFF].

	other's partial state.			

The root-identity probe in detail. These are the two generated root structs; run 2 read v1's slot 0 as `b`:

v1 (declares a, then b)

```
struct __pers_root {
    pmem_ptr<persistent<Box>> a;
    pmem_ptr<persistent<Box>> b;
};
```

v2 (declares b, then a): same pool, data swapped

```
struct __pers_root {
    pmem_ptr<persistent<Box>> b;
    pmem_ptr<persistent<Box>> a;
};
```

5 · Gap catalogue

Ordered by how each gap fails. A **silent** failure (wrong data, no error) is far worse than a compile error, because a compile error at least tells the user they wrote something unsupported.

#	gap	how it fails	breaks	needed for YCSB?
G1	Schema/root identity: position-based roots, no layout check across builds	**silent wrong data**	I6	yes (every rebuild)
G2	Library classes (`std::vector`, `std::map`, …) as fields	**silent, then a crash on restart**	I1 I2 I3	not in HashTable; common in real code
G3	Pointers other than `U*` (`U**`, `F*`, `void*`)	**silent stale address**	I2	**yes** (`HashNode** table`)
G4	Initialiser-list → assignment conversion: parameter shadowing (`size = size`), dropped default arguments; in-class default initialisers dropped [HANDOFF]	**silent wrong initial value**	correctness	likely. [decided 2026-10-20] fix: `this->` assignments
G5	No isolation; thread-unsafe root creation	**silent races**	I7	**yes** (multi-threaded)
G6	Dynamic arrays of scalars: `new int[n]`, and `char*`		I3	**yes** (`HashNode::value`)

	byte buffers (`memset` , `v[i] = c`)	compile error / rejected		
G7	Fixed-size array fields	compile error	—	no
G8	Inheritance, virtual functions, overloaded operators (skipped), static members, templated classes	compile error	—	no
G9	Free functions taking `T*` , and the persistent/volatile boundary (layouts are not compatible)	compile error; the user must retype parameters	—	partly
G10	Multi-TU programs: `__pers_root` is emitted "before main" in one file	link/compile error	—	**yes** (roots are in `ycsb_benchmark.cpp`)
G11	Root-level arrays (`HashTable** ht_node0`)	not discovered as roots	I1 I2	**yes**
G12	Writes from outside methods	runtime abort	I3 (by design)	no
G13	Permanent leaks on overwrite; no pool introspection or GC	slow pool exhaustion	I5	long runs
G14	Engineering: types matched by printed name (not namespace-aware); text splicing; no test suite; CMake files git-ignored	fragile	—	indirectly
G15	Cost model unmeasured: a transaction on every method (reads too), `pmemobj_direct` on every dereference, a snapshot per field write	performance	—	for the paper
G16	Compiler-generated copy assignment of `persistent<F>` bypasses `store()` (`max`	**silent unlogged write**	I3	yes. **Fixed locally** with one `operator=` (§12.5); not yet on GitHub `main`

	`= cur` ; whole-object copy) [demo]			

6 · How to make the tool complete

Most of the gaps share one root cause. The tool makes its decisions *while emitting text*, and any type it has no rule for falls through to a default that is silently wrong. Five structural changes, in a suggested order:

1. **A real analysis phase before rewriting.** Compute the persistence closure first: the persistent types, each field with its P(T), the methods, the `new` sites, and the functions that receive persistent pointers. Print it as a report the user can read. Then rewrite from that model. This makes the tool's decisions inspectable and testable, which a paper needs anyway (for "what did the compiler do to YCSB").
2. **Make P(T) total.** Every type gets either a defined rule or a diagnostic that stops the tool. There is no `/* unwrapped */` fall-through and no "skipping … (not found)". This one change turns G2, G3 and G7 from silent failures into explicit decisions:
   - pointer to pointer → nested `pmem_ptr` ;
   - pointer to scalar → `pmem_ptr<persistent<F>>` ;
   - fixed array → `persistent<F>[N]` ;
   - library class → error ("use a persistent container") until a persistent container exists;
   - union, `void*` or function pointer → error.
3. **Schema fingerprint in the pool.** At creation, store a hash of `__pers_root` and of every specialization's layout, and refuse to open a pool whose hash differs. Key roots by name rather than position, as a small name → OID table. This closes G1, the only silent failure that hits every rebuild. It is cheap: the analysis phase already has all the layouts.
4. **Semantics-preserving construction.** Emit `this->f = …` (or keep a real initialiser list on the specialization), and carry over default arguments and in-class initialisers. This closes G4.
5. **Testing infrastructure.**
   - Golden-output tests: the `concept/` files already are the expected outputs.
   - The probe programs from this review, as negative tests.
   - A crash-injection harness that aborts at each logged store and checks recovery. Tools in the pmreorder/Yat family are the systematic version; a simple `--crash-at=N` counter in `store()` gets most of the value.

After these, the language-coverage items (G6, G8–G11) are additions to a sound base rather than patches on a leaky one.

7 · Decisions only you can make

These are research positions rather than engineering tasks. Each changes what the tool should do.

1. **Is per-type persistence the thesis claim?** Today persistence is a static property of types, which mirrors introspective NUMA typing. The alternatives are persistence by *reachability at run time* (orthogonal persistence: whatever a root reaches at commit time is persisted) or by *allocation site*. Per-type is compile-time checkable and cheap, but it cannot follow `void*`, polymorphism or library internals. It also makes the persistent/volatile boundary a type error (G9). Deciding this decides how G2 and G9 should be handled.
   [decided 2026-10-20] **Per-type, and deliberately conservative: correctness first, and anything the tool cannot translate is rejected. `void*` and polymorphism are to be discussed later. Library internals are out of scope; in principle recursive specialization could reach them.**
2. **What is the failure-atomic unit?** Per outermost method call, today, is automatic and fits key-value operations. The alternatives are explicit `atomic {}` blocks (Mnemosyne) or lock-inferred sections (Atlas). The choice also determines the concurrency story.
   [decided 2026-10-20] **The outermost method call. Users may optionally wrap several calls in `transaction::run`, and PMDK flattens the nesting into one transaction.**
3. **Who provides isolation?** The tool could add a per-object lock to every specialization: automatic, but coarse and slow. Or the user's locks could be required to cover persistent methods, with the tool checking that (Atlas-style). Or the scope could be declared single-writer per object, which YCSB with per-table locks may already satisfy.
   [decided 2026-10-20] **A single writer per object for now; to be discussed in detail later.**
4. **What does a restart mean?** Today it means "re-run main against the existing data". Do you need user-level recovery hooks, such as rebuilding volatile indexes or re-initialising locks, or is re-running main the model?
   [decided 2026-10-20] **Re-running `main`. State that can be regenerated (hash state, zipfian and random generators) stays volatile. Keeping volatile configuration consistent with the persistent data it shapes (e.g. YCSB `--hash`) is the user's responsibility.**

5. **How should the program evolve with its data?** Refusing mismatched pools (§6.3) is the minimum. Supporting schema migration is a research topic in its own right. Decide whether it is in scope.
   [decided 2026-10-20] **Refuse mismatched pools for now.**
6. **Where is the evaluation boundary?** YCSB needs G3, G5, G6, G10 and G11. The pedagogical benchmarks need none of them. Choosing the target workload fixes the order of work.
   [decided 2026-10-20] **YCSB, with every rule kept general so that it does not overfit to one benchmark.**

8 · My assessment of the idea and the design

This is my reading as an informed outside reviewer, not a settled verdict. It rests on the code, on HANDOFF.md, on the probes in §4, and on the measurements in §10.

What is strong

1. **The programming model is the right one.** The user marks only the entry points to persistent data and the compiler derives the rest. The NVM literature has already argued that this is the usable level: AutoPersist (PLDI'19) states the case explicitly, that "programmers should only be required to identify durable roots". PersistentTyping reaches the same model *statically* and in an *unmanaged* language, C++, where AutoPersist and J-NVM needed a JVM. That combination is the project's clearest identity.
2. **Methods are a good failure-atomic unit, and there is a principled argument for it.** Methods are where a class establishes and restores its invariants, so making the outermost method call the transaction means that a crash always leaves objects in a state their own class considers valid. That is an object-oriented answer to "where do failure-atomic sections come from?", sitting between Mnemosyne's explicit blocks and Atlas's lock inference. The probe in §2 confirms the semantics end to end.
3. **Some guarantees hold by construction.** Deleting the mutable `operator T&` means an unlogged scalar write is a compile error rather than a bug. Restricting `pmem_ptr` fields to `persistent<U>*` makes most persistent → volatile links type errors. These are Corundum-style guarantees (ASPLOS'21) obtained in C++ by type translation.
4. **The output is ordinary, inspectable C++ over PMDK.** There is no new runtime, no VM, and no binary instrumentation. Recovery is PMDK's well-tested undo log. A user can read, debug and even hand-edit what the compiler produced. That is a real practical advantage over runtime-based systems, and a strong artifact-evaluation story.

5. **It reuses a technique you already published.** Introspective typing from the NUMA work now has a second, harder instance. Two instances turn a one-off trick into a *technique*, which is what a PL venue wants (see §11 for how far to push this without reopening the unification question).

What is weak, or risky

1. **The type closure is static, so it cannot see dynamic structure.** `void*`, polymorphic base pointers and library containers are invisible to it (G2, G8). AutoPersist avoids this by following reachability at run time. You need either a convincing argument that the static fragment covers realistic programs, or a fallback.
2. **Soundness rests on text splicing.** The probes found silent failures (G1–G4) in four separate places. For a paper that claims safety, "the tool rejects or correctly translates every program" has to be argued, and ideally proved for a core fragment. Today it is not.
3. **Two copies of every class, with no bridge between them.** `C` and `persistent<C>` coexist but cannot be passed to each other's code (G9), and their layouts differ. Code size grows with the number of persistent types. Interop with existing libraries is the question every reviewer will ask about C++.
4. **No isolation, while the target workload is concurrent.** YCSB is multi-threaded, and PMDK transactions do not isolate.
5. **The hardware story has changed.** Intel cancelled Optane in 2022. The field's current answer, which is persuasive, is that most of this work was always about *in-memory crash consistency*, and that CXL makes the problem live again, even for volatile shared memory. Desnoyers et al. argue that "the bulk of persistent-memory research has not in fact addressed memory persistence, but rather in-memory crash consistency". The paper's motivation has to be written in those terms.

9 · How it compares with existing systems

system (venue)	language	what the programmer marks	how persistent types arise	failure-atomic unit	main runtime cost
libpmemobj++ (PMDK)	C++	every field (`p<T>` , `persistent_ptr`), every allocation, every transaction	written by hand	explicit `transaction::run`	undo log + PMDK allocator
	C / C++		annotations	explicit atomic blocks	STM / logging

Mnemosyne, NV-Heaps (ASPLOS'11)		persistent variables/ regions, atomic blocks			
Atlas (OOPSLA'14)	C	persistent regions	allocation in a persistent region	inferred from lock-held sections	undo log on every store, at run time
AutoPersist (PLDI'19)	Java (modified JVM)	**durable roots only**	run-time reachability: objects move to NVM when they become reachable	failure-atomic regions	persistence checks and object movement in the JVM
J-NVM (SOSP'21)	Java	persistent classes, failure-atomic blocks	code generator splits each class into an off-heap data structure plus a volatile proxy	explicit blocks	proxy indirection
Corundum (ASPLOS'21)	Rust	persistent types and pools, explicitly	written by hand; the Rust type system enforces safety	explicit transactions	logging; reported at or below PMDK
CrashLang ("Towards Verifying Crash Consistency", PACMPL, Oct 2025)	new language	—	type system guarantees crash consistency of well-typed programs	language-defined	—
PersistentTyping	C++	**durable roots only** (`persistent<T>*` at file scope or in `main`)	**static** type-directed closure: generated `persistent<C>` specializations	outermost method call (automatic)	PMDK undo log + allocator; nothing at run time beyond PMDK

Where it sits. It shares AutoPersist's model (mark roots, derive the rest) and J-NVM's mechanism (generate persistent versions of user classes). It differs from both in being *static* and *C++*: there is no runtime reachability tracking, no JVM, and no proxy objects, and the output is plain PMDK code. Its safety goals overlap with Corundum's, but it

reaches them by translating types rather than asking the programmer to write persistent types. The novelty claim has to be made precisely against AutoPersist and J-NVM. Those are the two papers a reviewer will cite back at you.

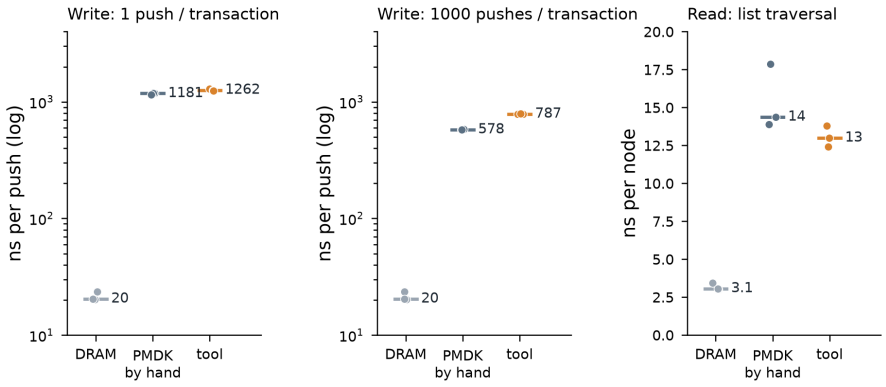
Sources checked for this table: AutoPersist (PLDI 2019), J-NVM (SOSP 2021), Corundum (ASPLOS 2021), CrashLang (Lee, Truong & Demsky

10 · Advantages and overheads, with measurements

Programmer effort (the main advantage)

For the stack benchmark, the user's file has **3** lines that mention persistence (an `#include`, the root declaration, and an optional `transaction::run`), out of 39 code lines. The hand-written libpmemobj++ version in `concept/stack/baseline_stack.cpp` has **27** such lines out of 75. That file also implements `pop()`, so this is an indication, not an exact like-for-like count. A paper would measure this systematically (lines changed, annotations, types touched) across all benchmarks, against PMDK and ideally Corundum.

Run-time cost, measured



Lower is better. Stack of 200k nodes; dots = 3 runs, bar + value = median. i9-9900K; pool on tmpfs with PMEM_IS_PMEM_FORCE=1 (software cost, not media latency).

Figure 5. Software overhead of the three variants on the same 200k-node stack (ecee-bilbo, i9-9900K). The pool is on tmpfs with PMEM_IS_PMEM_FORCE=1, so PMDK runs its persistent-memory flush path (clflushopt + sfence) without msync. This measures the CPU-side cost of persistence, not the latency of real PM or CXL media, which would add to both persistent variants.

operation	DRAM	hand-written PMDK	persist-tool output	tool vs PMDK
push, one transaction each (ns/op)	20	1181	1262	**1.07×**
push, 1000 per transaction (ns/op)	20	578	788	**1.36×**

traversal (ns/node)	3.1	14.4	13.0	**0.90×**

- **Against hand-written PMDK, the tool costs little where it matters.** One transaction per operation, the YCSB shape, is 7% slower. Reads are at parity: both translate an OID on every dereference, about 4× DRAM.
- **Batched writes are 36% slower, and the cause is visible in the generated code.** The generated `persistent<Node>` constructor opens its own nested transaction and *snapshots two fields of an object allocated in the same transaction* (`value = v; next = nullptr;` as logged assignments). Hand-written PMDK initialises them directly. Logging a fresh object is pure waste: PMDK already discards that allocation on abort. Emitting a real initialiser list, or unlogged stores for freshly allocated objects, should close most of the gap. This is the likely cause; the measurement does not isolate it.
- **Against DRAM, persistence costs roughly 58× per write operation.** That is the price of the transaction and the persistent allocator, and it is shared with any PMDK-based system. It is not specific to this tool.

Other overheads (not measured yet)

- **Read-only methods pay for a transaction.** Every generated method is wrapped, including `getCount` and `print`. In the traversal above one transaction is amortised over 200k nodes. A read-heavy workload such as YCSB-C, with one short `get` per operation, will pay the full begin/commit on every read. Not wrapping `const` methods, or methods with no writes, is a simple and valuable analysis.
- **Memory.** A pointer becomes 16 bytes instead of 8, so `Node` grows from 16 to 24 bytes. libpmemobj++ has the same cost. PMDK's per-allocation header is extra in both.
- **Code size and compile time.** Each persistent class exists twice, and the tool runs two Clang passes. Both are probably small, but they should be reported.
- **One global pool** of fixed size; no per-structure pools.

11 · Is it publishable, and where?

> **Short answer.** Yes, there is a publishable paper here, but not yet at OOPSLA. Today it is a solid workshop paper, at NVMW (already your target) or DIMES. An OOPSLA or PLDI paper needs three things it does not have yet: a precise novelty claim against AutoPersist and J-NVM, a soundness argument for the translation, and an evaluation on a real, concurrent workload.

> [decided 2026-10-20] **Chosen framing.** Thesis umbrella: angle 3, "Introspective typing for memory semantics". Main paper: angle 1, "Durable roots for C++, by static type translation", with angle 2 (methods as failure-atomic sections) folded in as the *time* half of the model. Angle 4 (systems/CXL) may fork later.

Candidate angles (strongest first)

1. **"Durable roots for C++, by static type translation."** This is the AutoPersist usability claim, delivered at compile time in an unmanaged language, with no runtime. The core contribution is the translation P(T), stated as a definition over C++ types, with a theorem for a core fragment: if the tool accepts a program, the result satisfies placement, stable references and logged writes (I1–I3 in §4). The completeness work in §6 is exactly what makes that theorem true. Fits **OOPSLA / PLDI**.
2. **"Methods as failure-atomic sections."** Object invariants give automatic, intuitive atomicity without annotations or lock inference. Pair it with a static check that persistent state changes only inside these sections, and evaluate atomicity granularity against Mnemosyne-style blocks and Atlas-style lock inference. This can be a section of paper 1, or its own short paper.
3. **"Introspective typing for memory semantics."** Present NUMA placement and persistence as two instances of one compiler technique: a recursive, type-directed derivation of memory-semantic variants of user types. This is the most "PL" framing and the strongest for OOPSLA. It does *not* require building the unified `mem<T, …>` type you decided against; the paper can describe the shared technique while the tools stay separate. It may suit a journal or thesis chapter better if you want to keep this paper focused on persistence.
4. **Systems angle.** Crash consistency for CXL-attached or shared memory, with this tool as the programming model. Fits **ASPLOS**, but needs a hardware or CXL story you do not have yet.

Venue fit

venue	fit	what it needs beyond today
NVMW, DIMES (workshops)	good now	The current state, plus the overhead numbers and the gap analysis.
CGO, CC	good after the completeness work	A robust tool (§6 items 1–2, 4), YCSB, overhead vs PMDK, and compile-time numbers. Less formalism needed.

OOPSLA, PLDI	plausible after significant work	Angle 1 or 3. A formal P(T) with a soundness argument; the static vs reachability-based design argued against AutoPersist; the generated-class mechanism against J-NVM; a usability evaluation (lines and annotations vs PMDK and Corundum); crash-consistency validation; concurrency handled or explicitly scoped. OOPSLA has a record of PM programming-model papers (Atlas'14; Raad et al.'s persistency semantics'19/'20; CrashLang appeared in PACMPL in October 2025, likely the OOPSLA issue).
ASPLOS	only with a hardware or CXL angle	Real PM or CXL experiments, and multi-threading.

Objections a reviewer will raise, and the answer each needs

1. **"How is this different from AutoPersist?"** Static versus dynamic: no run-time persistence checks or object moves, and C++ rather than a managed runtime. You must also own the cost of that choice: no `void*`, polymorphism or STL. Show what fraction of real code falls inside the fragment.
2. **"J-NVM already generates persistent versions of user classes."** Answer with recursive closure from roots (J-NVM persists the classes you mark), automatic method-level atomicity, and no proxy indirection on access.
3. **"Is the translation sound?"** A theorem for a core calculus, plus the probe-style negative tests showing the tool rejects what it cannot translate.
4. **"Optane is dead."** The in-memory crash-consistency and CXL argument (§8, item 5), with emulated-PM numbers clearly labelled as software cost.
5. **"What about concurrency?"** Either a design (per-object locks or lock-coverage checking), or a clearly stated single-writer scope that YCSB with per-table locks satisfies.
6. **"Overhead vs hand-written code?"** Today: 7% on per-operation writes, parity on reads. Fix the constructor logging and the read-only transactions before submission.

A paper outline this could become

1. Motivation: crash consistency is hard even when the hardware works; programmers should name only the roots.
2. Programming model: `persistent<T>*` roots; per-method atomicity; what is and is not supported.
3. The translation P(T) and the generated specializations; soundness for a core fragment.
4. Implementation: Clang LibTooling, the two passes, the analysis phase.

5. Evaluation:
    - effort: lines and annotations vs PMDK and Corundum;
    - performance: data structures plus YCSB A–F vs DRAM and hand-written PMDK;
    - crash-injection validation;
    - coverage of real code.
6. Related work, built around the table in §9.

12 · Decisions and discussion (2026-10-20)

12.1 Decision log

topic	decision
Target workload	YCSB (single table → full harness), with rules kept general so as not to overfit
What persists	Per type, conservatively: translate or reject; no silent fall-through
Failure-atomic unit	Outermost method call on a persistent object; optional user-level grouping with `transaction::run`
Isolation	Single writer per object for now (YCSB's per-table harness locks are held across the whole call, so they are released after commit)
Schema evolution	Refuse mismatched pools
Strings	**Drop `pmem_string`**. No current benchmark uses strings: in the repo's YCSB, `HashNode::key` is `uint64_t` and `value` is a byte buffer
Fixed-size arrays	Support now: `T x[N]` → `P(T) x[N]`
Bulk memory operations	Persistent `memset` / `memcpy` helpers later (one snapshot per range)
Library classes (`std::vector`, …)	Out of scope for now; reject. `std::` functions (e.g. `swap`) are not a concern for now
Inheritance / `virtual`	Inheritance is of interest; polymorphism deferred, because persistent types are never cast to regular types and YCSB needs neither
Volatile configuration	The user's responsibility (e.g. keep YCSB's `--hash` fixed across runs)
Constructor rewrite	`this->field = expr` in the transaction-wrapped body, in declaration order; keep default arguments and in-class initialisers

Copy assignment	Add `operator=(const persistent&)` routed through `store()` (done locally)
Publication	Thesis: introspective typing for memory semantics. Paper: durable roots for C++ + methods as failure-atomic sections

12.2 Pointer vocabulary

Each name has the form "*where the pointer lives*-resident *kind* pointer". The one invariant the tool enforces: **a pointer that lives in pmem must be a pool offset**.

name	lives in	points to	stored as	after restart
regular-resident regular pointer	DRAM	DRAM	raw address	n/a (ordinary C++)
regular-resident pmem pointer	DRAM	pmem object	raw address (`persistent<T>*` locals and parameters)	dies with the process; valid within one run
pmem-resident pmem pointer	pmem	pmem	pool offset (`pmem_ptr`)	✓ the only kind allowed in the pool
pmem-resident regular pointer	pmem	anything	raw address	✗ never valid; the tool must never emit one

A raw address in pmem that points into pmem is also a pmem-resident regular pointer: the pool may map at a different address next run. It can pass tests by luck when the pool happens to land at the same address, so restart tests should not be relied on to catch it. An object's vtable pointer is an *implicit* pmem-resident regular pointer to code, which is why polymorphism needs care later (§12.4).

12.3 The revised type translation P(T)

With `pmem_string` gone, P is defined by structural recursion on the C++ type, with one pointer rule that covers every pointer depth:

```
P(F)        = persistent<F>                F fundamental (and enums: extend the primitive s
P(C)        = persistent<C>                C user class; generate the specialization recurs
P(T*)       = pmem_ptr< P(T) >             any pointee: a pmem-resident pmem pointer
P(T[N])     = P(T) [N]                     fixed-size array, part of the enclosing object
P(other)    = error                        library classes, unions, void*, references,
                                            function pointers, virtual methods (for now)
```

field	P(field)	YCSB use
`Node* next`	`pmem_ptr<persistent<Node>>`	`HashNode::next`
`HashNode** table`	`pmem_ptr<pmem_ptr<persistent<HashNode>>>`	`HashTable::table`
`char* value`	`pmem_ptr<persistent<char>>` (dynamic array; its length is known only where it is allocated)	`HashNode::value`
`int bins[4]`	`persistent<int> bins[4]`	(Histogram-style)

Two details make this work. Because `sizeof(persistent<F>) == sizeof(F)` (4 bytes for `int`, measured), arrays of wrapped scalars keep the original layout and indexing. And because only `persistent<F>` and `pmem_ptr` remain, a soundness argument for P has only two library types to reason about.

Consequences:

- The string-based benchmarks (`kvnode`, `kvnode2`, `kvstr`) will be rejected loudly (`strcpy(key, …)` no longer type-checks) until the bulk helpers exist. Keep them as tests that must be rejected.
- YCSB's `update()` writes the payload one byte at a time, which logs each byte. Hoisting one range snapshot out of such loops is a natural compiler optimisation, and the automatic form of the memset/memcpy helpers.

12.4 Answers to the questions raised

question	answer
Why does root order matter, if everything is static?	It is static *within* one build, but the pool outlives the binary. `__pers_root` is read by byte offset using the current build's struct layout, and no names are stored. **Fix:** pass a fingerprint (a hash of the root fields and every specialization's field names, types and order) as the PMDK *layout* string. `pmemobj_open` already refuses a pool whose layout string differs.
Does `__pers_root` need updating after pass 2?	No. It holds only the *entry points*. Every other object is reached by following pmem-resident pmem pointers from them, like a garbage collector's root set. Pass 2 changes types, not roots, and it *reads* `__pers_root` to seed its traversal.
Isn't the constructor already in a transaction?	Yes. Atomicity and placement are fine. The bug is C++ name lookup: inside the body, `size` means the parameter, so the field is never written (§12.5). Construction is in fact covered by the *enclosing* transaction (`pmem_get_or_create` or the calling method), so either fix would preserve atomicity.

Do users have to wrap calls in transactions?	No. Each generated method opens its own. A user's `transaction::run` around several calls is optional; PMDK flattens the nesting, so the whole group commits or aborts together, and it is faster (787 vs 1262 ns per push in §10).
Why is `contents` a problem if `store()` logs?	`pmemobj_tx_add_range_direct` is not a transaction. It registers a range with the active transaction (it snapshots the old bytes and marks them to be flushed at commit). Users never name `contents`, but the *compiler-generated* copy assignment wrote it directly without calling `store()` (§12.5). An unlogged write breaks both atomicity (nothing to roll back) and durability (commit flushes only logged ranges). Abort-based crash tests catch only the first.
Single pool: what is the trade-off?	For: any pointer can reach any object; every transaction covers all data (PMDK transactions cannot span pools); one root and one recovery step. Against: size fixed at creation; no per-structure lifecycle; one failure domain; and it collides with NUMA placement (one pool per node would split transactions). Recommendation: keep one pool, and make its size a runtime parameter.
Strings vs byte buffers?	The bytes are alike; the difference is how the length is known. YCSB's `value` is filled with `memset(value, k & 0xFF, …)`, so for every key that is a multiple of 256 it is all zero bytes. Read as a string it is empty. Settled by dropping strings: `char*` is just a dynamic array (§12.3).
Inheritance?	Mirror the class hierarchy (`persistent<D> : persistent<B>`). Making a class persistent pulls in its bases, which is easy statically. Derived classes stored through a base pointer need a conservative whole-program over-approximation. `virtual` is deferred: today the signature builder drops `virtual` silently, so a class with virtual methods should get an explicit "not supported" error.
Is I5 the inverse of I1?	Essentially, yes: I1 is "reachable ⇒ in pool" and I5 is the converse, "in pool ⇒ reachable". Together, the pool holds exactly the persistent object graph. Breaking I1 is a correctness bug; breaking I5 is a leak.

12.5 Two silent bugs confirmed during the discussion [demo]

Constructor rewrite (G4). A plain-C++ stand-in of the generated shape (wrapper with default value 0; initialiser list turned into body assignments):

```
struct Orig {
    int size = 0;
    int capacity = 16;                 // in-class default initialiser
    Orig(int size = 0) : size(size) {} // default argument + initialiser list
};
// The shape the tool generates today: fields wrapped, init list -> body assignment,
// default argument and in-class initialiser dropped.
```

```
struct Gen {
    Wrap size;
    Wrap capacity;
    Gen(int size) { size = size; }
};
```

```
original : size=5 capacity=16
generated: size=0 capacity=0
```

Calling the generated constructor with no argument fails: `error: no matching constructor for initialization of 'Gen'`. So all three effects are real: shadowing (5 → 0), the dropped in-class initialiser (16 → 0), and the dropped default argument.

Copy assignment of `persistent<F>` (G16). An exact copy of the primitive specialization's operators at `64bb1ab`, with the PMDK snapshot replaced by a counter. Before:

```
max = cur + 0   (int rhs)    max 0 -> 7   store() calls: 1
max += cur                   max 0 -> 7   store() calls: 1
max = cur       (wrapped rhs) max 0 -> 7   store() calls: 0
h = g                        store() calls: 0   (whole-object copy)
```

With the one line you added, `persistent& operator=(const persistent& o) { store(o.load()); return *this; }`:

```
max = cur + 0   (int rhs)    max 0 -> 7   store() calls: 1
max += cur                   max 0 -> 7   store() calls: 1
max = cur       (wrapped rhs) max 0 -> 7   store() calls: 1
h = g                        store() calls: 2   (whole-object copy)
```

The whole-object copy is fixed too, because the generated class's default copy assignment assigns member by member and so reaches the new operator. Copy and move constructors need nothing: a copy creates a new object in a transactional allocation, and declaring a copy assignment makes moves fall back to it. **Status:** added in your local tree; GitHub `main` is still at `64bb1ab`.

12.6 Revised plan toward YCSB

1. **Foundations** (each turns a silent failure into correct output or an explicit error):
    - `this->` constructor assignments in declaration order, with default arguments and in-class initialisers kept;
    - the copy assignment fix (done locally);
    - `contents` made private (tidiness);
    - the layout fingerprint as the PMDK layout string;

    - P(T) total, with errors for everything outside §12.3;
    - `pmem_string` removed.
2. **Pointers at any depth.** The single rule P(T*) = `pmem_ptr<P(T)>`; `new T[n]` placed in the pool; element access rewritten. This covers `HashNode** table` and `char* value`.
3. **Fixed-size arrays.** `P(T)[N]`.
4. **Roots for YCSB.** Root arrays (`HashTable** ht_node0`) and a root shared across translation units (`ycsb_benchmark.cpp` + `main.cpp`).
5. **Initialisation.** Document single-threaded initialisation, which YCSB already does with thread 0 and a barrier.
6. **Performance.** No transaction for read-only methods (matters for YCSB-C); range snapshots hoisted out of byte loops.
7. **Evaluation.** YCSB A–F against DRAM and a hand-written PMDK port, plus crash-injection tests on emulated pmem.

Appendix · How the evidence was produced

Eight probe programs (six probes, with root identity tested by three versions), written as ordinary user code, were run through the current tool at `64bb1ab` (`root-setup`, then `recurse-persistent`), compiled with `clang++ -lpmemobj`, and executed on bilbo. Each probe had its own pool, which was a regular file because `/mnt/pmem-emu` was not mounted. That changes how PMDK flushes, but not its transaction or recovery semantics. The stack example and the baseline results for the six existing benchmarks come from the earlier test run of the same commit. The overhead benchmark (§10) used three hand-written variants of one stack program, compiled with `-O2`; the persistent variant was produced by the same tool build.

```
build rc=0
=== atomic
[atomic] compile rc=0 errors=0
  $ ./prog  -> exit 0
    a=2 b=2
  $ ./prog crash -> exit 134

  $ ./prog print -> exit 0
    a=3 b=3
=== fixedarr
[fixedarr] compile rc=1 errors=1
=== stl
[stl] compile rc=0 errors=0
  $ ./prog  -> exit 0
    n=2: 7 8
  $ ./prog  -> exit 139
```

```
=== inherit
[inherit] compile rc=1 errors=2
=== mainwrite
[mainwrite] compile rc=0 errors=0
  $ ./prog  -> exit 134

=== root identity (one pool shared by v1 -> v2 -> v3)
[rootv1] compile rc=0 errors=0
  $ ./prog  -> exit 0
    a=1 b=2
  $ ./prog  -> exit 0
    a=1 b=2
[rootv2] compile rc=0 errors=0
  $ ./prog  -> exit 0
    a=2 b=1
[rootv3] compile rc=0 errors=0
  $ ./prog  -> exit 0
    a=0 b=0
```